COS 487 / 587 Course Project Part 2: IR Rangers

Esther Greene
University of Southern Maine
Portland, Maine, USA
esther.greene@maine.edu

Courtney Jackson
University of Southern Maine
Portland, Maine, USA
courtney.jackson@maine.edu

Nathaniel Serrano
University of Southern Maine
Portland, Maine, USA
nathaniel.serrano@maine.edu

Abstract

Keywords

information retrieval, BM25, SPLADE, dense retrieval, rank fusion, scientific document retrieval, passage retrieval

1 Introduction

The volume of scientific literature makes finding relevant work increasingly difficult, both at the level of whole papers and of the specific passages that answer a question. In the first phase of this project, we built a retrieval system for arXiv papers and paragraphs[1] using two term-matching models, and found that BM25 significantly outperformed TF-IDF on both tasks. Our analysis also exposed the limits of exact term matching: hard queries that paraphrase technical terminology in general language, for example asking about "wrist-worn optical sensors" rather than PPG. share few terms with the document that answer them, and BM25 often failed on them, especially for paragraphs.

This paper describes the second phase, in which we add two neural retrieval models intended to bridge this vocabulary gap: a learned sparse model, SPLADE [4], which represents text as weights over vobcablulary terms, including related terms the text does not contain, and a dense bi-encoder [8], which represents text as a single embedding vector. We evaluate each model both as released and after fine-tuning on our Train queries, rank papers by their best-scoring paragraph [3], and combine all three models with reciprocal rank fusion [2]. BM25, our best model from part 1, serves as the baseline throughout.

The test collection consists of 34,308 arXiv papers published in August and September 2024. Each paper is provided both as raw HTML scraped from the ar5iv website and as a processed JSON file containing its identifier, title, abstract, and paragraphs. Because the JSON files were produced with rule-based extraction, they contain noise such as leftover LaTeX markup and merged words, and display equations are missing altogether. In total, the collection contains 2,275,574 paragraphs, an average of about 66 per paper.

We address two tasks over this collection: *paper retrieval*, which returns papers identified by their arXiv ID, and *paragraph retrieval*, which returns individual paragraphs. For each query, systems return their top 100 results. Three query sets are provided: Train (2,000 queries), Study (250 queries with relevance judgements, which we use for all analysis in this paper), and Test (250 queries whose judgments are withheld). Each set contains 30% easy, 50% medium, and 25% hard queries. Relevance is graded on a three-point scale: a document that fully addresses the query is relevant (2), one that provides useful information without fully addressing it is partially relevant (1), and any document not in the judgments is considered non-relevant.

[1]Code: https://github.com/esthergreene/IR-Rangers

SPLADE is the most effective single model on both tasks. On papers it improves nDCG@5 over BM25 from 0.514 to 0.631; on paragraphs its advantage is smaller and not significant. The dense model outperforms BM25 on papers but not on paragraphs, and fine-tuning did not significantly improve either neural model. Fusing all three models gives the best results, with nDCG of 0.701 on papers and 0.550 on paragraphs, significantly above both BM25 and SPLADE alone. Our analysis shows that the neural models retrieve paragraphs that paraphrase the query, but still rank highly paragraphs that share the query's vobcabulary in an unrelated field.

2 Methodology

Our system retrieves with three first-stage models that share the query sets, run format, and evaluation pipeline. BM25 (Section 2.2) searches inverted indexes over separately preprocessed paper and paragraph collections, as in Part 1. The two neural models, SPLADE (Section 2.4) and a dense bi-encoder (Section 2.5), receive more lightly preprocessed text (Section 2.1) and index paragraphs only. Both start from publically released checkpoints, which we evaluate as released and after fine-tuning on Train queries (Section 2.3). Because the neural indexes contain only paragraphs, papers are ranked from their paragraphs' scores (Section 2.6). Finally, runs from different models can be combined with reciprocal rank fusion (Section 2.7). Section 2.8 describes our evaluation setup.

2.1 Preprocessing

BM25 and the neural models receive differently preprocessed text, because they match text in different ways.

For BM25, we keep the Part 1 pipeline unchanged. Inline mathematics, remaining LaTeX commands, URLs, and numeric citation markers are removed; text is lowercased and split into alphanumeric tokens; tokens shorter than two characters or consisting only of digits are discarded; stopwords are removed using NLTK's English list [1], extended with terms common in scientific writing such as "et", "al", and "fig"; and the remaining tokens are reduced with the Snowball stemmer [7]. Cleaning reduces the paragraph collection from 168.0 to 114.9 million tokens (31.6%) and leaves 8,389 paragraphs, almost all standalone formulas, empty. These are excluded, leaving 2,267,185 indexed paragraphs. Both encoders trucate their input at 256 WordPiece tokens.

2.2 BM25 Baseline

Our baseline is the BM25 model from Part 1, unchanged. It scores a document d for a query q as

$$\text{score}(q,d) = \sum_{t \in q} idf_t \cdot \frac{tf_{t,d}\,(k_1+1)}{tf_{t,d} + k_1\left(1 - b + b\,L_d/L_{avg}\right)}, \tag{1}$$

$$idf_t = \ln\left(1 + \frac{N - df_t + 0.5}{df_t + 0.5}\right), \tag{2}$$

where N is the number of documents, df_t the number containing term t, $tf_{t,d}$ the occurrences of t in d, and L_d and L_{avg} the length of d and the average length, in tokens. This is the "Lucene (accurate)" variant of Kamphuis et al. [5], whose IDF remains positive for terms that occur in more than half of the documents. The parameter k_1 controls how quickly repeated occurrences of a term saturate, and b how strongly scores are normalized by document length. Scoring is term-at-a-time over the postings of the query terms, vectorized with NumPy.

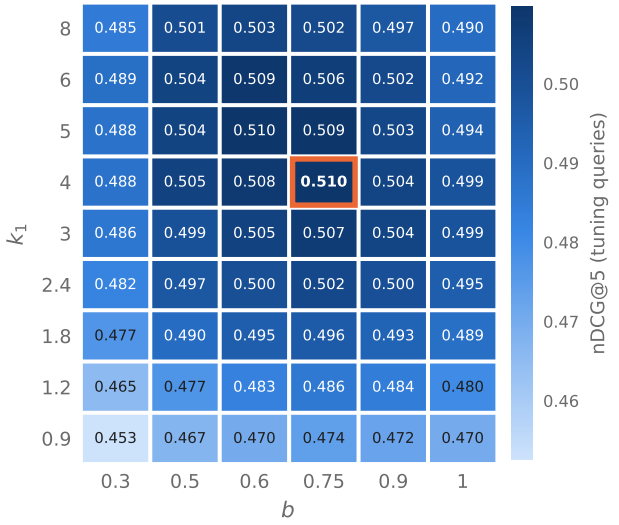
We tuned k_1 and b separately for each collection by grid search on 1,600 Train queries, selecting the setting with the highest nDCG@5 (Figure 1). The selected settings, $k_1 = 4.0$, $b = 0.75$ for papers and $k_1 = 0.4$, $b = 0.9$ for paragraphs, also outperformed both the textbook values ($k_1 = 1.2$, $b = 0.75$) [6] and Anserini's defaults ($k_1 = 0.9$, $b = 0.4$) [9] on the remaining 400 Train queries (nDCG@5 of 0.473 vs. 0.454 and 0.436 on papers, and 0.371 vs. 0.354 and 0.344 on paragraphs). The tenfold difference in k_1 reflects document length: in a paper averaging 3,361 tokens, repeated occurrences of a term indicate its topic, whereas paragraphs, with a median of 40 tokens, rarely repeat a term, so whether a term occurs at all matters more.

2.3 Training Data

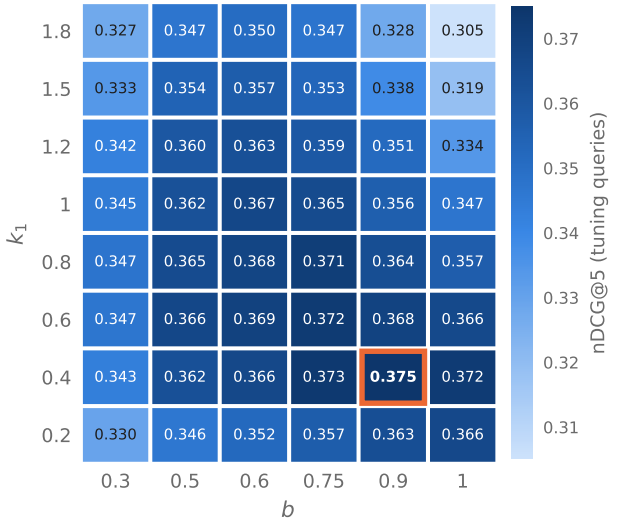
We fine-tune both neural models on Train queries only; Study and Test queries are never used for training or model selection. The 2,000 Train queries are split by query into 1,800 training and 200 validation queries, stratified by difficulty so that both sets keep the 30/50/20 mix of easy, medium, and hard queries, and no validation query appears in training. Every paragraph judged relevant or partially relevant serves as a positive, giving 5,138 positives for the training queries and 582 for the validation queries, about 2.9 per query.

The Train judgments contain no non-relevant (grade 0) paragraphs, so negatives must be mined. We use BM25 hard negatives: unjudged paragraphs that BM25 retrieves for the query, which share vocabulary and topic with it and are therefore more informative than random paragraphs. Because judgments are incomplete, an unjudged paragraph near the top of BM25's ranking is often relevant, so we skip BM25's top 20 results and sample 30 negatives per query at random from ranks 21–100, with a fixed seed per query. Random sampling gives each query a mix of close and easier negatives rather than only the hardest ones. Each training example is a (query, positive, negative) triple, one per positive, cycling through the query's negatives. Both models are trained with a contrastive loss in which each query's positive must outscore its own hard negative as well as the positives and negatives of the other queries in the batch (Sections 2.4 and 2.5).

Both models select their best checkpoint by nDCG@10 on the validation queries. Encoding all 2.28 million paragraphs at every evaluation would take 30 to 60 minutes (Section 3.7), so validation instead ranks a fixed sample of the collection: every validation positive, the top 100 BM25 results for each validation query, and 10,000 random paragraphs. Scores on this sample are used only to compare checkpoints, not to report effectiveness.



(a) Papers.



(b) Paragraphs.

Figure 1: BM25 nDCG@5 on 1,600 Train tuning queries. Outlined cells mark the selected settings.

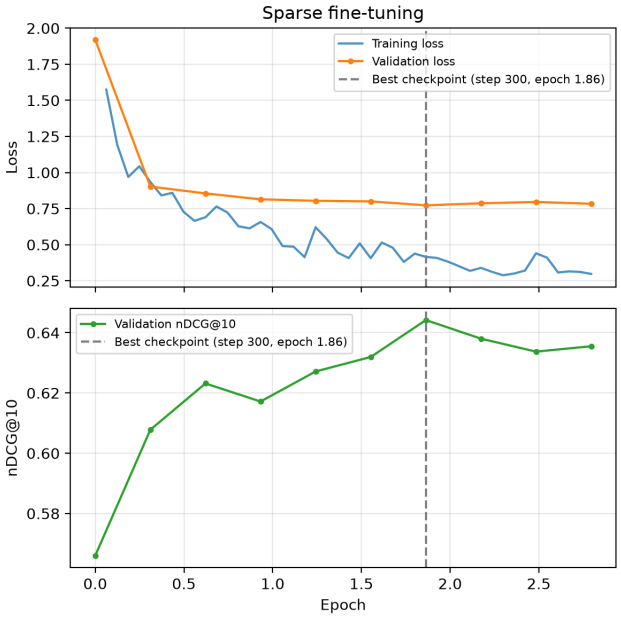
2.4 Learned Sparse Retrieval

2.5 Dense Retrieval

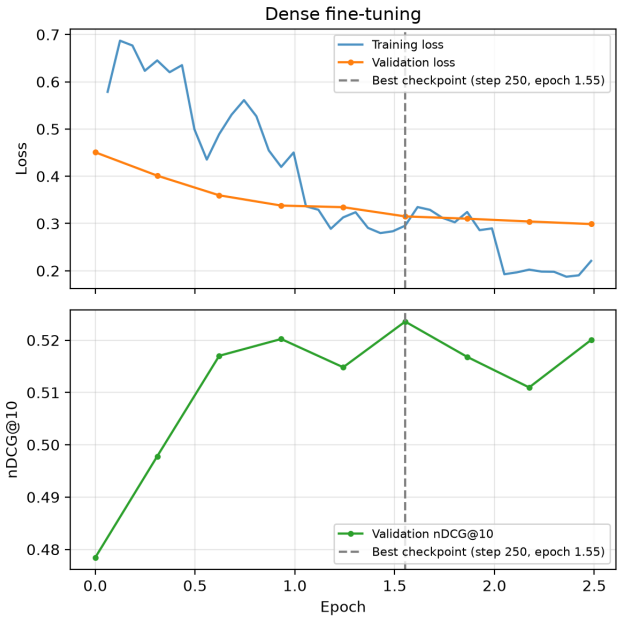
2.6 Paper Ranking

2.7 Rank Fusion

$$\mathrm{RRF}(d) = \sum_{m \in M} \frac{1}{k + \mathrm{rank}_m(d)}, \qquad k = 60, \tag{3}$$



(a) SPLADE.



(b) Dense (MiniLM).

Figure 2: Training and validation loss.

2.8 Evaluation Setup
3 Evaluation Results
3.1 Training
3.2 Effectiveness
3.3 Effect of Fine-Tuning
3.4 Rank Fusion
3.5 Query Difficulty
3.6 Test Annotation
3.7 Efficiency
4 Results Analysis
4.1 Comparing the Neural Models
4.2 Comparison with the BM25 Baseline
4.3 Successful Retrieval Cases
4.4 Failure Cases
4.5 Why Fine-Tuning Gave Little
4.6 Why Fusion Helps
4.7 Strengths and Weaknesses
5 Conclusion

References

[1] Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural Language Processing with Python*. O'Reilly Media.

[2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 758–759.

[3] Zhuyun Dai and Jamie Callan. 2019. Deeper Text Understanding for IR with Contextual Neural Language Modeling. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 985–988.

[4] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. 2021. SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2288–2292.

[5] Chris Kamphuis, Arjen P. de Vries, Leonid Boytsov, and Jimmy Lin. 2020. Which BM25 Do You Mean? A Large-Scale Reproducibility Study of Scoring Variants. In *Advances in Information Retrieval: 42nd European Conference on IR Research (ECIR 2020)*. Springer, 28–34.

[6] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press. https://nlp.stanford.edu/IR-book/

[7] Martin F. Porter. 2001. Snowball: A Language for Stemming Algorithms. https://snowballstem.org/texts/introduction.html.

[8] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 3982–3992.

[9] Peilin Yang, Hui Fang, and Jimmy Lin. 2017. Anserini: Enabling the use of lucene for information retrieval research. In *Proceedings of the 40th international ACM SIGIR conference on research and development in information retrieval*. 1253–1256.

Table 1: Effectiveness on the 250 Study queries. Strict metrics count only grade-2 documents (238 queries). Superscripts mark a significant improvement over the model with that letter (paired t-test, $p < 0.05$).

	Model	nDCG	nDCG@5	MRR	P@5	P@10	R@100	Strict MRR	Strict P@5
Papers									
a	BM25	0.582	0.514	0.544	0.137	0.080	0.803	0.535	0.124
b	Dense	0.640^{a}	0.592^{a}	0.631^{a}	0.156^{a}	0.084	0.807	0.606^{a}	0.141^{a}
c	SPLADE	$\mathbf{0.671}^{a}$	$\mathbf{0.631}^{ab}$	$\mathbf{0.658}^{a}$	$\mathbf{0.167}^{ab}$	$\mathbf{0.090}^{ab}$	**0.825**	$\mathbf{0.645}^{a}$	$\mathbf{0.151}^{ab}$
Paragraphs									
a	BM25	0.474	0.416	0.516	0.174	0.107	0.592	0.450	0.115
b	Dense	0.447	0.377	0.477	0.175	0.104	0.607	0.415	0.115
c	SPLADE	$\mathbf{0.494}^{b}$	$\mathbf{0.434}^{b}$	$\mathbf{0.539}^{b}$	**0.190**	$\mathbf{0.116}^{b}$	$\mathbf{0.630}^{a}$	$\mathbf{0.484}^{b}$	$\mathbf{0.135}^{ab}$

Table 2: Zero-shot versus fine-tuned neural models on Study queries. † marks a significant difference between the two versions of a model (paired t-test, $p < 0.05$).

	Papers					Paragraphs				
Model	nDCG	nDCG@5	MRR	P@5	R@100	nDCG	nDCG@5	MRR	P@5	R@100
Dense, zero-shot	0.635	0.591	0.619	$0.166^{\dagger}$	0.812	0.458	0.385	0.484	0.174	0.633
Dense, fine-tuned	0.640	0.592	0.631	0.156	0.807	0.447	0.377	0.477	0.175	0.607
SPLADE, zero-shot	0.668	0.625	0.654	0.161	0.815	0.485	0.431	0.524	0.186	0.603
SPLADE, fine-tuned	0.671	0.631	0.658	0.167	0.825	0.494	0.434	0.539	0.190	0.630

Table 3: Reciprocal rank fusion on Study queries. Superscripts mark a significant improvement over the run with that letter (paired t-test, $p < 0.05$).

	Run	nDCG	nDCG@5	MRR	P@5	P@10	R@100	Strict MRR	Strict P@5
Papers									
a	BM25	0.582	0.514	0.544	0.137	0.080	0.803	0.535	0.124
b	SPLADE	0.671^{a}	0.631^{a}	0.658^{a}	0.167^{a}	0.090^{a}	0.825	0.645^{a}	0.151^{a}
c	RRF(BM25+SPLADE)	0.675^{a}	0.628^{a}	0.659^{a}	0.170^{a}	0.094^{a}	0.834^{a}	0.646^{a}	0.149^{a}
d	RRF(all three)	$\mathbf{0.701}^{abc}$	$\mathbf{0.655}^{ac}$	$\mathbf{0.689}^{ac}$	$\mathbf{0.174}^{a}$	$\mathbf{0.095}^{ab}$	$\mathbf{0.853}^{abc}$	$\mathbf{0.673}^{ac}$	$\mathbf{0.154}^{a}$
Paragraphs									
a	BM25	0.474	0.416	0.516	0.174	0.107	0.592	0.450	0.115
b	SPLADE	0.494	0.434	0.539	0.190	0.116	0.630^{a}	0.484	0.135^{a}
c	RRF(BM25+SPLADE)	0.531^{ab}	0.468^{ab}	0.569^{a}	0.201^{a}	0.119^{a}	0.666^{ab}	0.508^{a}	0.136^{a}
d	RRF(all three)	$\mathbf{0.550}^{abc}$	$\mathbf{0.489}^{abc}$	$\mathbf{0.582}^{ab}$	$\mathbf{0.218}^{abc}$	$\mathbf{0.130}^{abc}$	$\mathbf{0.689}^{abc}$	$\mathbf{0.523}^{ab}$	$\mathbf{0.148}^{abc}$

Table 5: Our judgments of top-ranked Test paragraphs.

Grade	Dense	SPLADE	Both
Relevant (2)	TODO	TODO	TODO
Partially relevant (1)	TODO	TODO	TODO
Non-relevant (0)	TODO	TODO	TODO

Table 6: Index size, indexing time, startup time and average query latency on the 250 Study queries (papers / paragraphs). All times were measured on one shared server with an Intel Core i7-11700 CPU (2.50 GHz), 31 GiB of RAM and an NVIDIA RTX A5000 GPU (24 GB); encoders run on the GPU and index search on the CPU. Query latency includes query processing and, for neural paper runs, retrieving 2,000 paragraphs and MaxP. Neural models index only paragraphs. With a checkpoint every 50,000 paragraphs, dense indexing took 70.1 minutes.

Model	Index	Indexing	Startup (s)	Query (ms)
BM25	242 MB / 974 MB	81 s / 175 s	15.6 / 34.6	7.3 / 62.4
Dense	3.3 GB	31.8 min	64.5 / 41.0	231.6 / 230.1
SPLADE	2.8 GB	63.4 min	97.3 / 69.1	536.5 / 490.9